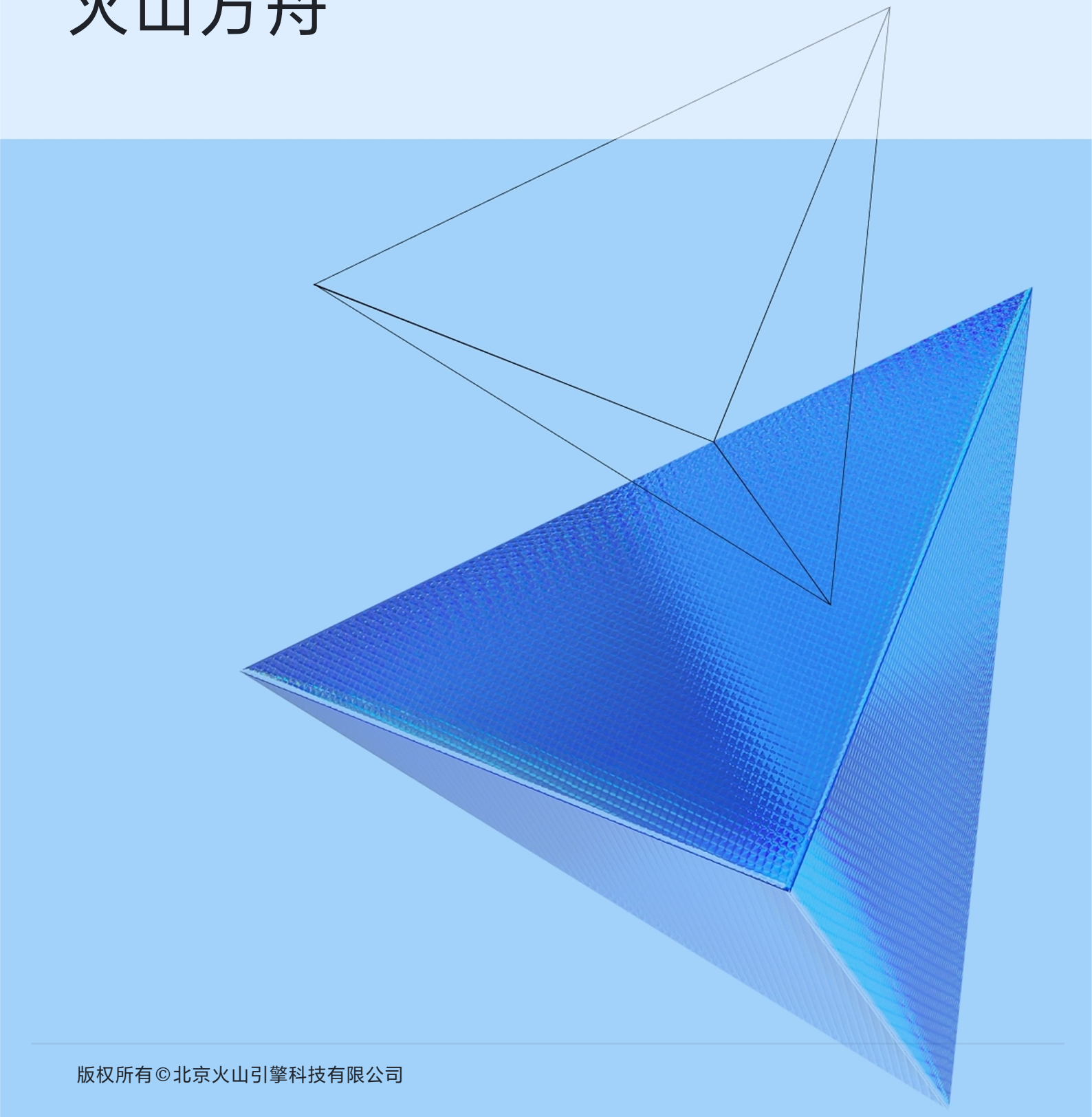
火山引擎

快速入门

火山方舟



版权所有©北京火山引擎科技有限公司

法律声明

本《火山方舟》的所有内容，包括但不限于文字、商标、架构、图示、图片、页面布局等，其知识产权（著作权、商标权、专利权、商业秘密等）归属于北京火山引擎科技有限公司及其关联公司（火山引擎），非经火山引擎书面同意，任何个人和组织不得复制、使用、修改、转发或以任何违反本《火山方舟》所承载的目的进行传播。

本《火山方舟》陈述内容仅作为产品的通用性介绍和参考性指引，火山引擎保留按“现状”和“当前可用”的形式提供产品和服务的权利。火山引擎不对本《火山方舟》中所载的产品功能、性质、质量、标准等内容进行明示或默示的保证和承诺，最终以您与火山引擎实际签署的协议为准。

如您发现本《火山方舟》有任何错误或歧义，或发现有对本《火山方舟》、产品本身的侵权行为，请与火山引擎取得联系。

联系方式：service@volcengine.com，400-850-0030
（周一至周五 10:00-18:00）

目录

法律声明----------1
目录----------1
1. 快速入门----------1

版权所有©北京火山引擎科技有限公司

1. 快速入门

数分钟内完成你的首次 API 调用。

快速入门(新手版)
为零基础用户设计的快速入门

方舟文档 MCP
极简配置，协同 AI 完成大模型开发

Agent Plan
一站式智能体开发方案，快速搭建专属 AI Agent

1 获取并配置 API Key

1. 获取 API Key：在控制台左下角选择 API Key 管理，创建你的 API Key。
2. 配置环境变量：在终端中运行下面命令（替换 `your_api_key_here` 为你的方舟API Key），配置 API Key 到环境变量。

配置持久化环境变量方法参见 环境变量配置指南。

MacOS

```Bash
export ARK_API_KEY="your_api_key_here"
```

Linux

```Bash
export ARK_API_KEY="your_api_key_here"
```

Windows_CMD

```Bash
setx ARK_API_KEY "your_api_key_here"
```

Windows_PowerShell

版权所有©北京火山引擎科技有限公司

```powershell
$env:ARK_API_KEY = "your_api_key_here"
```

2 开通模型服务

访问 开通管理页面 开通模型服务。

3 安装 SDK

安装官方或三方 SDK。

Python

> 运行环境中需安装 Python 版本 3.7 或以上。

- 安装方舟 SDK：

```bash
pip install 'volcengine-python-sdk[ark]'
```

> Windows CMD 不支持单引号作为引用符。如在 Windows CMD 中执行，请改用双引号：`pip install "volcengine-python-sdk[ark]"`

- 安装 OpenAI SDK：

```bash
pip install openai
```

Go

> 环境中安装 Go 版本 1.18 或以上。

在代码中通过下方方法引入 Go SDK

```go
import (
  "github.com/volcengine/volcengine-go-sdk"
)
```

Java

> 环境中安装 Java 版本 1.8 或以上。

在项目的 `pom.xml` 文件中添加以下依赖配置。

XML

```
<dependency>
 <groupId>com.volcengine</groupId>
 <artifactId>volcengine-java-sdk-ark-runtime</artifactId>
 <version>LATEST</version>
</dependency>
```

4 发起 API 请求

以下按输入输出类别列举的典型任务，选择任意示例代码体验如何通过 API 调用及体验大模型及方舟平台能力。

文本生成

传入文本类信息给模型，进行问答、分析、改写、摘要、编程、翻译等任务，并返回文本结果。

输入	输出预览
Hello	· 思考： Got it, let's see. The user said "hello". I need to respond in a friendly and welcoming way. Since the system prompt mentions a professional but friendly tone, I should keep it natural. Maybe something like "Hello! How can I assist you today?" That's simple, polite, and open-ended to encourage the user to share what they need help with. · 回答： Hello! How can I assist you today? Whether you have a question, need help with something specific, or just want to chat, feel free to let me know. 😊

Python

```
import os
from volcenginesdkarkruntime import Ark

client = Ark(
    base_url='https://ark.cn-beijing.volces.com/api/v3',
    api_key=os.getenv('ARK_API_KEY'),
)

response = client.responses.create(
    model="doubao-seed-2-1-pro-260628",
    input="hello", # Replace with your prompt
```

```python
    # thinking={"type": "disabled"}, #  Manually disable deep thinking
)

for item in response.output:
    if item.type == "reasoning":
        for summary in item.summary:
            if summary.type == "summary_text":
                print(f"思考：{summary.text}")
    elif item.type == "message":
        for content in item.content:
            if content.type == "output_text":
                print(f"回答：{content.text}")
```

Curl

```bash
curl https://ark.cn-beijing.volces.com/api/v3/responses \
  -H "Authorization: Bearer $ARK_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "model": "doubao-seed-2-1-pro-260628",
    "input": "hello"
  }'
```

· 关闭深度思考：配置 `"thinking":{"type": "disabled"}` 。

Go

```go
package main

import (
    "context"
    "fmt"
    "os"
    "github.com/volcengine/volcengine-go-sdk/service/arkruntime"
    "github.com/volcengine/volcengine-go-sdk/service/arkruntime/model/responses"
)

func main() {
    client := arkruntime.NewClientWithApiKey(
        // Get API Key：https://console.volcengine.com/ark/region:cn-beijing/apikey
        os.Getenv("ARK_API_KEY"),
        arkruntime.WithBaseUrl("https://ark.cn-beijing.volces.com/api/v3"),
```

```go
Go )
    ctx := context.Background()

    resp, err := client.CreateResponses(ctx, &responses.ResponsesRequest{
        Model: "doubao-seed-2-1-pro-260628",
        Input: &responses.ResponsesInput{Union: &responses.ResponsesInput_StringValue{StringValue:
"hello"}}, // Replace with your prompt
        // Thinking: &responses.ResponsesThinking{Type: responses.ThinkingType_disabled.Enum()}, //
Manually disable deep thinking
    })
    if err != nil {
        fmt.Printf("response error: %v\n", err)
        return
    }
    fmt.Println(resp)
}
```

Java

```java

package com.ark.sample;

import com.volcengine.ark.runtime.service.ArkService;
import com.volcengine.ark.runtime.model.responses.request.*;
import com.volcengine.ark.runtime.model.responses.response.ResponseObject;

public class demo {
    public static void main(String[] args) {
        String apiKey = System.getenv("ARK_API_KEY");
        // The base URL for model invocation
        ArkService arkService = ArkService.builder().apiKey(apiKey).baseUrl("https://ark.cn-
beijing.volces.com/api/v3").build();

        CreateResponsesRequest request = CreateResponsesRequest.builder()
                .model("doubao-seed-2-1-pro-260628")
                .input(ResponsesInput.builder().stringValue("hello").build()) // Replace with your prompt
                // .thinking(ResponsesThinking.builder().type(ResponsesConstants.THINKING_TYPE_DISABLED).b
Manually disable deep thinking
                .build();

        ResponseObject resp = arkService.createResponse(request);
        System.out.println(resp);

        arkService.shutdownExecutor();
```

```
Java}
  }
```

OpenAI SDK

```python
import os
from openai import OpenAI

client = OpenAI(
    base_url='https://ark.cn-beijing.volces.com/api/v3',
    api_key=os.getenv('ARK_API_KEY'),
)

response = client.responses.create(
    model="doubao-seed-2-1-pro-260628",
    input="hello", # Replace with your prompt
    extra_body={
        # "thinking": {"type": "disabled"}, #  Manually disable deep thinking
    },
)

for item in response.output:
    if item.type == "reasoning":
        for summary in item.summary:
            if summary.type == "summary_text":
                print(f"思考：{summary.text}")
    elif item.type == "message":
        for content in item.content:
            if content.type == "output_text":
                print(f"回答：{content.text}")
```

- 文本生成：文本生成使用指南。
- 深度思考：深度思考能力使用指南。
- 迁移至 Responses API：新用户推荐，更简洁的上下文管理能力、强大的工具调用能力。
- Chat API：存量业务迭代推荐，广泛使用的 API。

多模态理解

传入图片、视频、PDF文件给模型，进行分析、内容审核、问答、视觉定位等基于多模态理解相关任务，并返回文本结果。

版权所有©北京火山引擎科技有限公司

输入	输出预览
	· 思考：用户现在需要找支持输入图片的模型系列，看表格里的输入列中的图像列，哪个模型对应的图像输入是√。看表格，Doubao-1.5-vision那一行的输入图像列是√，其他两个Doubao-1.5-pro和lite的输入图像都是 × ，所以答案是Doubao-1.5-vision。 · 回答：支持输入图片的模型系列是Doubao-1.5-vision

> 支持输入图片的模型系列是哪个？

Python

```python
import os
from volcenginesdkarkruntime import Ark

client = Ark(
    base_url='https://ark.cn-beijing.volces.com/api/v3',
    api_key=os.getenv('ARK_API_KEY'),
)

response = client.responses.create(
    model="doubao-seed-2-1-pro-260628",
    input=[
        {
            "role": "user",
            "content": [
                {
                    "type": "input_image",
                    "image_url": "https://ark-project.tos-cn-beijing.volces.com/doc_image/ark_demo_img_1.png"
                },
                {
                    "type": "input_text",
                    "text": "支持输入图片的模型系列是哪个？"
                },
            ],
        }
    ]
)

for item in response.output:
```

```python
    if item.type == "reasoning":
        for summary in item.summary:
            if summary.type == "summary_text":
                print(f"思考：{summary.text}")
    elif item.type == "message":
        for content in item.content:
            if content.type == "output_text":
                print(f"回答：{content.text}")
```

Curl

```bash
curl https://ark.cn-beijing.volces.com/api/v3/responses \
-H "Authorization: Bearer $ARK_API_KEY" \
-H 'Content-Type: application/json' \
-d '{
    "model": "doubao-seed-2-1-pro-260628",
    "input": [
        {
            "role": "user",
            "content": [
                {
                    "type": "input_image",
                    "image_url": "https://ark-project.tos-cn-beijing.volces.com/doc_image/
ark_demo_img_1.png"
                },
                {
                    "type": "input_text",
                    "text": "支持输入图片的模型系列是哪个？"
                }
            ]
        }
    ]
}'
```

Go

```go
package main

import (
    "context"
    "fmt"
    "os"
```

```
    "github.com/samber/lo"
    "github.com/volcengine/volcengine-go-sdk/service/arkruntime"
    "github.com/volcengine/volcengine-go-sdk/service/arkruntime/model/responses"
)

func main() {
    client := arkruntime.NewClientWithApiKey(
        // Get API Key：https://console.volcengine.com/ark/region:cn-beijing/apikey
        os.Getenv("ARK_API_KEY"),
        arkruntime.WithBaseUrl("https://ark.cn-beijing.volces.com/api/v3"),
    )
    ctx := context.Background()

    inputMessage := &responses.ItemInputMessage{
        Role: responses.MessageRole_user,
        Content: []*responses.ContentItem{
            {
                Union: &responses.ContentItem_Image{
                    Image: &responses.ContentItemImage{
                        Type:    responses.ContentItemType_input_image,
                        ImageUrl: lo.ToPtr("https://ark-project.tos-cn-beijing.volces.com/doc_image/
ark_demo_img_1.png"),
                    },
                },
            },
            {
                Union: &responses.ContentItem_Text{
                    Text: &responses.ContentItemText{
                        Type: responses.ContentItemType_input_text,
                        Text: "支持输入图片的模型系列是哪个？",
                    },
                },
            },
        },
    }

    resp, err := client.CreateResponses(ctx, &responses.ResponsesRequest{
        Model: "doubao-seed-2-1-pro-260628",
        Input: &responses.ResponsesInput{
            Union: &responses.ResponsesInput_ListValue{
                ListValue: &responses.InputItemList{ListValue: []*responses.InputItem{{
                    Union: &responses.InputItem_InputMessage{
                        InputMessage: inputMessage,
                    },
                }}},
            },
        },
```

```go
    })
    if err != nil {
        fmt.Printf("response error: %v\n", err)
        return
    }
    fmt.Println(resp)
}
```

Java

```java
package com.ark.sample;

import com.volcengine.ark.runtime.service.ArkService;
import com.volcengine.ark.runtime.model.responses.request.*;
import com.volcengine.ark.runtime.model.responses.content.*;
import com.volcengine.ark.runtime.model.responses.item.*;
import com.volcengine.ark.runtime.model.responses.response.ResponseObject;
import com.volcengine.ark.runtime.model.responses.constant.ResponsesConstants;

public class demo {
  public static void main(String[] args) {
    String apiKey = System.getenv("ARK_API_KEY");
    ArkService arkService = ArkService.builder().apiKey(apiKey).baseUrl("https://ark.cn-beijing.volces.com/api/v3")
        .build();

    CreateResponsesRequest request = CreateResponsesRequest.builder()
        .model("doubao-seed-2-1-pro-260628")
        .input(ResponsesInput.builder().addListItem(
            ItemEasyMessage.builder().role(ResponsesConstants.MESSAGE_ROLE_USER).content(
                MessageContent.builder()
                    .addListItem(InputContentItemImage.builder()
                        .imageUrl("https://ark-project.tos-cn-beijing.volces.com/doc_image/ark_demo_img_1.png").build())
                    .addListItem(InputContentItemText.builder().text("支持输入图片的模型系列是哪个？ ").build())
                    .build())
                .build())
            .build())
        .build();
    ResponseObject resp = arkService.createResponse(request);
    System.out.println(resp);

    arkService.shutdownExecutor();
```

```
Java
    }
}
```

OpenAI SDK

```
Python
import os
from openai import OpenAI

client = OpenAI(
    base_url='https://ark.cn-beijing.volces.com/api/v3',
    api_key=os.getenv('ARK_API_KEY'),
)

response = client.responses.create(
    model="doubao-seed-2-1-pro-260628",
    input=[
        {
            "role": "user",
            "content": [
                {
                    "type": "input_image",
                    "image_url": "https://ark-project.tos-cn-beijing.volces.com/doc_image/
ark_demo_img_1.png"
                },
                {
                    "type": "input_text",
                    "text": "支持输入图片的模型系列是哪个？"
                },
            ],
        }
    ]
)

for item in response.output:
    if item.type == "reasoning":
        for summary in item.summary:
            if summary.type == "summary_text":
                print(f"思考：{summary.text}")
    elif item.type == "message":
        for content in item.content:
            if content.type == "output_text":
                print(f"回答：{content.text}")
```

· 多模态理解：多模态理解详细使用指南。

- 视觉定位 Grounding：图片中找到对应目标并返回坐标任务。
- GUI Agent 能力：在计算机/移动设备中完成自动化任务。
- 文件输入(File API)：传入图片、视频、文档接口。

图片生成

传入图片、文字给模型，进行以下场景&任务：

- 广告、海报、组图等图片生成；
- 增改元素、颜色更换等图片编辑；
- 油墨、水墨等风格切换。

提示词	输出预览
充满活力的特写编辑肖像，模特眼神犀利，头戴雕塑感帽子，色彩拼接丰富，眼部焦点锐利，景深较浅，具有Vogue杂志封面的美学风格，采用中画幅拍摄，工作室灯光效果强烈。	

Python

```python
import os
# Install SDK:  pip install 'volcengine-python-sdk[ark]'
from volcenginesdkarkruntime import Ark

client = Ark(
    # The base URL for model invocation
    base_url="https://ark.cn-beijing.volces.com/api/v3",
    # Get API Key：https://console.volcengine.com/ark/region:cn-beijing/apikey
    api_key=os.getenv('ARK_API_KEY'),
)

imagesResponse = client.images.generate(
    # Replace with Model ID
    model="doubao-seedream-5-0-260128",
    prompt="充满活力的特写编辑肖像，模特眼神犀利，头戴雕塑感帽子，色彩拼接丰富，眼部焦点锐利，景深较浅，具有Vogue杂志封面的美学风格，采用中画幅拍摄，工作室灯光效果强烈。",
    size="2K",
    response_format="url",
```

```python
    watermark=False
)

print(imagesResponse.data[0].url)
```

Curl

```bash
curl https://ark.cn-beijing.volces.com/api/v3/images/generations \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $ARK_API_KEY" \
  -d '{
    "model": "doubao-seedream-5-0-260128",
    "prompt": "充满活力的特写编辑肖像，模特眼神犀利，头戴雕塑感帽子，色彩拼接丰富，眼部焦点锐利，景深较浅，具有Vogue杂志封面的美学风格，采用中画幅拍摄，工作室灯光效果强烈。",
    "size": "2K",
    "watermark": false
}'
```

· 您可按需替换 Model ID。Model ID 查询见 模型列表 。

Java

```java
package com.ark.sample;

import com.volcengine.ark.runtime.model.images.generation.*;
import com.volcengine.ark.runtime.service.ArkService;
import okhttp3.ConnectionPool;
import okhttp3.Dispatcher;
import java.util.concurrent.TimeUnit;

public class ImageGenerationsExample {
    public static void main(String[] args) {
        // Get API Key：  https://console.volcengine.com/ark/region:cn-beijing/apikey
        String apiKey = System.getenv("ARK_API_KEY");
        ConnectionPool connectionPool = new ConnectionPool(5, 1, TimeUnit.SECONDS);
        Dispatcher dispatcher = new Dispatcher();
        ArkService service = ArkService.builder()
                .baseUrl("https://ark.cn-beijing.volces.com/api/v3") // The base URL for model invocation
                .dispatcher(dispatcher)
                .connectionPool(connectionPool)
                .apiKey(apiKey)
                .build();
```

```
Java
        GenerateImagesRequest generateRequest = GenerateImagesRequest.builder()
                .model("doubao-seedream-5-0-260128") // Replace with Model ID
                .prompt("充满活力的特写编辑肖像，模特眼神犀利，头戴雕塑感帽子，色彩拼接丰富，眼部焦点锐利，景深较浅，具有Vogue杂志封面的美学风格，采用中画幅拍摄，工作室灯光效果强烈。")
                .size("2K")
                .sequentialImageGeneration("disabled")
                .responseFormat(ResponseFormat.Url)
                .stream(false)
                .watermark(false)
                .build();
        ImagesResponse imagesResponse = service.generateImages(generateRequest);
        System.out.println(imagesResponse.getData().get(0).getUrl());

        service.shutdownExecutor();
    }
}
```

Go

```
package main

import (
    "context"
    "fmt"
    "os"
    "strings"
    "github.com/volcengine/volcengine-go-sdk/service/arkruntime"
    "github.com/volcengine/volcengine-go-sdk/service/arkruntime/model"
    "github.com/volcengine/volcengine-go-sdk/volcengine"
)

func main() {
    client := arkruntime.NewClientWithApiKey(
        os.Getenv("ARK_API_KEY"), // Get API Key：https://console.volcengine.com/ark/region:cn-beijing/apikey
        arkruntime.WithBaseUrl("https://ark.cn-beijing.volces.com/api/v3"), // The base URL for model invocation
    )
    ctx := context.Background()

    generateReq := model.GenerateImagesRequest{
        Model:        "doubao-seedream-5-0-260128", // Replace with Model ID
```

```Go
    Prompt:      "充满活力的特写编辑肖像，模特眼神犀利，头戴雕塑感帽子，色彩拼接丰富，眼部焦点锐利，景深较浅，具有Vogue杂志封面的美学风格，采用中画幅拍摄，工作室灯光效果强烈。",
    Size:        volcengine.String("2K"),
    ResponseFormat: volcengine.String(model.GenerateImagesResponseFormatURL),
    Watermark:    volcengine.Bool(false),
  }

  imagesResponse, err := client.GenerateImages(ctx, generateReq)
  if err != nil {
    fmt.Printf("generate images error: %v\n", err)
    return
  }

  fmt.Printf("%s\n", *imagesResponse.Data[0].Url)
}
```

OpenAI

```Python
import os
from openai import OpenAI

client = OpenAI(
  base_url="https://ark.cn-beijing.volces.com/api/v3", # The base URL for model invocation
  api_key=os.getenv('ARK_API_KEY'), # Get API Key：https://console.volcengine.com/ark/region:cn-beijing/apikey
)

imagesResponse = client.images.generate(
  # Replace with Model ID
  model="doubao-seedream-5-0-260128",
  prompt="充满活力的特写编辑肖像，模特眼神犀利，头戴雕塑感帽子，色彩拼接丰富，眼部焦点锐利，景深较浅，具有Vogue杂志封面的美学风格，采用中画幅拍摄，工作室灯光效果强烈。",
  size="2K",
  response_format="url",
  extra_body={
    "watermark": False,
  },
)

print(imagesResponse.data[0].url)
```

- Doubao Seedream 4.0-5.0 教程：主流生图模型能力以及如何通过 API 调用。
- Seedream 4.0-5.0 提示词指南：使用生图模型时，如何编写提示词。

版权所有©北京火山引擎科技有限公司

视频生成

通过文本描述、图像素材、视频素材，快速生成高质量、风格多样的视频内容。

提示词	输出画面预览
一位身穿绿色亮片礼服的女性站在粉红色背景前，周围飘落着五彩斑斓的彩纸	

Python

```
import os
import time
# Install SDK:  pip install 'volcengine-python-sdk[ark]'
from volcenginesdkarkruntime import Ark

client = Ark(
    base_url="https://ark.cn-beijing.volces.com/api/v3", # The base URL for model invocation
    api_key=os.environ.get("ARK_API_KEY"), # Get API Key：  https://console.volcengine.com/ark/region:cn-beijing/apikey
)

if __name__ == "__main__":
    print("----- create request -----")
    create_result = client.content_generation.tasks.create(
        model="doubao-seedance-2-0-260128", # Replace with Model ID
        content=[
            {
                # Combination of text prompt and parameters
                "type": "text",
                "text": "一位身穿绿色亮片礼服的女性站在粉红色背景前，周围飘落着五彩斑斓的彩纸 --wm true --dur 5"
            }
        ]
    )
    print(create_result)

    # Polling query section
```

Python
```python
print("----- polling task status -----")
task_id = create_result.id
max_wait = 600  # Maximum wait time in seconds
interval = 10  # Initial polling interval
start = time.time()

while time.time() - start < max_wait:
    get_result = client.content_generation.tasks.get(task_id=task_id)
    status = get_result.status
    if status == "succeeded":
        print("----- task succeeded -----")
        print(get_result)
        break
    elif status == "failed":
        print("----- task failed -----")
        print(f"Error: {get_result.error}")
        break
    else:
        print(f"Current status: {status}, retrying after {interval:.0f}s...")
        time.sleep(interval)
        interval = min(interval * 1.5, 30)
else:
    raise TimeoutError(f"Task {task_id} timed out after {max_wait}s. "
                       f"Task may still be running - check status manually.")
```

Java

```java
package com.ark.sample;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.TimeUnit;
import com.volcengine.ark.runtime.model.content.generation.*;
import
com.volcengine.ark.runtime.model.content.generation.CreateContentGenerationTaskRequest.Conte
nt;
import com.volcengine.ark.runtime.service.ArkService;

public class ContentGenerationTaskExample {
  public static void main(String[] args) {
    String apiKey = System.getenv("ARK_API_KEY");
    ArkService service = ArkService.builder()
        .baseUrl("https://ark.cn-beijing.volces.com/api/v3") // The base URL for model invocation
        .apiKey(apiKey)
```

```
        .build();

    System.out.println("----- create request -----");
    List<Content> contents = new ArrayList<>();
    contents.add(Content.builder()
        .type("text")
        .text("一位身穿绿色亮片礼服的女性站在粉红色背景前，周围飘落着五彩斑斓的彩纸 --wm
true --dur 5")
        .build());

    // Create a video generation task
    CreateContentGenerationTaskRequest createRequest =
CreateContentGenerationTaskRequest.builder()
        .model("doubao-seedance-2-0-260128") // Replace with Model ID
        .content(contents)
        .build();

    CreateContentGenerationTaskResult createResult =
service.createContentGenerationTask(createRequest);
    System.out.println(createResult);

    // Get the details of the task
    String taskId = createResult.getId();
    GetContentGenerationTaskRequest getRequest = GetContentGenerationTaskRequest.builder()
        .taskId(taskId)
        .build();

    System.out.println("----- polling task status -----");
    while (true) {
      try {
        GetContentGenerationTaskResponse getResponse =
service.getContentGenerationTask(getRequest);
        String status = getResponse.getStatus();
        if ("succeeded".equalsIgnoreCase(status)) {
          System.out.println("----- task succeeded -----");
          System.out.println(getResponse);
          service.shutdownExecutor();
          break;
        } else if ("failed".equalsIgnoreCase(status)) {
          System.out.println("----- task failed -----");
          System.out.println("Error: " + getResponse.getStatus());
          service.shutdownExecutor();
          break;
        } else {
          System.out.printf("Current status: %s, Retrying in 3 seconds...\n", status);
          TimeUnit.SECONDS.sleep(3);
        }
```

```java
        } catch (InterruptedException ie) {
            Thread.currentThread().interrupt();
            System.err.println("Polling interrupted");
            service.shutdownExecutor();
            break;
        }
    }
  }
}
```

Go

```go
package main

import (
    "context"
    "fmt"
    "os"
    "time"
    "github.com/volcengine/volcengine-go-sdk/service/arkruntime"
    "github.com/volcengine/volcengine-go-sdk/service/arkruntime/model"
    "github.com/volcengine/volcengine-go-sdk/volcengine"
)

func main() {
    client := arkruntime.NewClientWithApiKey(
        os.Getenv("ARK_API_KEY"), // Get API Key：  https://console.volcengine.com/ark/region:cn-beijing/apikey
        arkruntime.WithBaseUrl("https://ark.cn-beijing.volces.com/api/v3"), // The base URL for model invocation
    )
    ctx := context.Background()
    // Replace with Model ID
    modelEp := "doubao-seedance-2-0-260128"

    fmt.Println("----- create request -----")
    createReq := model.CreateContentGenerationTaskRequest{
        Model: modelEp,
        Content: []*model.CreateContentGenerationContentItem{
            {
                Type: model.ContentGenerationContentItemTypeText,
                Text: volcengine.String("一位身穿绿色亮片礼服的女性站在粉红色背景前，周围飘落着五彩斑斓的彩纸 --wm true --dur 5"),
            },
```

```go
        },
    }
    createResp, err := client.CreateContentGenerationTask(ctx, createReq)
    if err != nil {
        fmt.Printf("create content generation error: %v", err)
        return
    }
    taskID := createResp.ID
    fmt.Printf("Task Created with ID: %s", taskID)

    // Polling query section
    fmt.Println("----- polling task status -----")
    for {
        getReq := model.GetContentGenerationTaskRequest{ID: taskID}
        getResp, err := client.GetContentGenerationTask(ctx, getReq)
        if err != nil {
            fmt.Printf("get content generation task error: %v", err)
            return
        }

        status := getResp.Status
        if status == "succeeded" {
            fmt.Println("----- task succeeded -----")
            fmt.Printf("Task ID: %s \n", getResp.ID)
            fmt.Printf("Model: %s \n", getResp.Model)
            fmt.Printf("Video URL: %s \n", getResp.Content.VideoURL)
            fmt.Printf("Completion Tokens: %d \n", getResp.Usage.CompletionTokens)
            fmt.Printf("Created At: %d, Updated At: %d", getResp.CreatedAt, getResp.UpdatedAt)
            return
        } else if status == "failed" {
            fmt.Println("----- task failed -----")
            if getResp.Error != nil {
                fmt.Printf("Error Code: %s, Message: %s", getResp.Error.Code, getResp.Error.Message)
            }
            return
        } else {
            fmt.Printf("Current status: %s, Retrying in 3 seconds... \n", status)
            time.Sleep(3 * time.Second)
        }
    }
}
```

- 视频生成教程：学习如何使用模型的视频生成能力，包括文本生成视频、首尾帧生视频、首帧生成视频等。
- Seedance-1.0-pro&pro-fast 提示词指南：使用生视频模型时，如何编写提示词。

工具使用

通过工具/插件让模型具体读取外部数据及函数的能力，包括

- 内置工具：联网搜索、图片处理、知识库检索等已集成至方舟平台的工具。
- 三方工具：兼容MCP 的三方工具。
- 自定义工具：您自行定义及开发的工具。

输入	输出预览
What's the weather like in Beijing?	According to the latest weather data as of March 10, 2026, the current weather in Beijing is sunny with a gentle wind (less than level 3). The temperature around 11:30 AM is approximately 9° C, and it is expected to reach a high of 12° C during the day. The weather will remain clear at night with a low temperature of 1° C. **Source** : Weather forecasts updated on March 10, 2026, from the Central Meteorological Observatory. **Note: Data is accurate as of the latest available update at 05:30 AM on March 10.**

Python

```
import os
from volcenginesdkarkruntime import Ark

client = Ark(
    base_url="https://ark.cn-beijing.volces.com/api/v3",
    api_key=os.getenv("ARK_API_KEY"),
)

response = client.responses.create(
    model="doubao-seed-2-1-pro-260628",
    input=[{"role": "user", "content": "What's the weather like in Beijing?"}],
    tools=[
        {
            "type": "web_search",
            "max_keyword": 2,
        }
    ],
)

for item in response.output:
```

```python
    if item.type == "message":
        for content in item.content:
            if content.type == "output_text":
                print(content.text)
```

Curl

```bash
curl https://ark.cn-beijing.volces.com/api/v3/responses \
-H "Authorization: Bearer $ARK_API_KEY" \
-H 'Content-Type: application/json' \
-d '{
    "model": "doubao-seed-2-1-pro-260628",
    "stream": true,
    "tools": [
        {
            "type": "web_search",
            "max_keyword": 3
        }
    ],
    "input": [
        {
            "role": "user",
            "content": [
                {
                    "type": "input_text",
                    "text": "What is the weather like in Beijing?"
                }
            ]
        }
    ]
}'
```

Java

```java
package com.ark.sample;
import com.fasterxml.jackson.core.JsonProcessingException;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.databind.JsonNode;
import com.volcengine.ark.runtime.model.responses.item.*;
import com.volcengine.ark.runtime.model.responses.request.*;
import com.volcengine.ark.runtime.model.responses.response.ResponseObject;
import com.volcengine.ark.runtime.model.responses.constant.ResponsesConstants;
```

Java

```java
import com.volcengine.ark.runtime.model.responses.content.InputContentItemText;
import com.volcengine.ark.runtime.model.responses.tool.*;
import com.volcengine.ark.runtime.service.ArkService;
import java.util.Arrays;
import java.util.List;

public class demo {
    public static ObjectMapper om = new ObjectMapper();

    public demo() throws JsonProcessingException {
    }

    public static List<ResponsesTool> buildTools() {
        ToolWebSearch t = ToolWebSearch.builder().build();
        System.out.println(Arrays.asList(t));
        return Arrays.asList(t);
    }

    public static void main(String[] args) throws JsonProcessingException {
        String apiKey = System.getenv("ARK_API_KEY");

        ArkService arkService = ArkService.builder().apiKey(apiKey).baseUrl("https://ark.cn-beijing.volces.com/api/v3").build();
        CreateResponsesRequest req = CreateResponsesRequest.builder()
                .model("doubao-seed-2-1-pro-260628")
                .input(ResponsesInput.builder().addListItem(
                        ItemEasyMessage.builder().role(ResponsesConstants.MESSAGE_ROLE_USER).content(
                                MessageContent.builder()
                                        .addListItem(InputContentItemText.builder().text("What's the weather like in Beijing?").build())
                                        .build()
                        ).build()
                ).build())
                .tools(buildTools())
                .build();
        ResponseObject resp = arkService.createResponse(req);
        System.out.println(resp);

        arkService.shutdownExecutor();
    }
}
```

Go

```go
package main
import (
 "context"
 "fmt"
 "os"
 "github.com/volcengine/volcengine-go-sdk/service/arkruntime"
 "github.com/volcengine/volcengine-go-sdk/service/arkruntime/model/responses"
)

func main() {
 client := arkruntime.NewClientWithApiKey(
  os.Getenv("ARK_API_KEY"),
  arkruntime.WithBaseUrl("https://ark.cn-beijing.volces.com/api/v3"), // The base URL for model invocation
 )
 ctx := context.Background()
 maxToolCalls := int64(1) // Limit the number of tool calls, adjust it according to your needs.

 inputMessage := &responses.ItemInputMessage{
  Role: responses.MessageRole_user,
  Content: []*responses.ContentItem{
   {
    Union: &responses.ContentItem_Text{
     Text: &responses.ContentItemText{
      Type: responses.ContentItemType_input_text,
      Text: "What's the weather like in Beijing?",
     },
    },
   },
  },
 }

 req := &responses.ResponsesRequest{
  Model: "doubao-seed-2-1-pro-260628",
  Input: &responses.ResponsesInput{
   Union: &responses.ResponsesInput_ListValue{
    ListValue: &responses.InputItemList{ListValue: []*responses.InputItem{{
     Union: &responses.InputItem_InputMessage{
      InputMessage: inputMessage,
     },
    }}}},
  },
  Tools: []*responses.ResponsesTool{
   {
    Union: &responses.ResponsesTool_ToolWebSearch{
```

```go
Go          ToolWebSearch: &responses.ToolWebSearch{
              Type: responses.ToolType_web_search,
            },
          },
        },
      },
      MaxToolCalls: &maxToolCalls,
    }

    resp, err := client.CreateResponses(ctx, req)
    if err != nil {
      fmt.Printf("Error: %v\n", err)
      os.Exit(1)
    }

    fmt.Printf("Response: %v\n", resp)
}
```

OpenAI SDK

```python
import os
from openai import OpenAI

client = OpenAI(
    base_url="https://ark.cn-beijing.volces.com/api/v3",
    api_key=os.getenv("ARK_API_KEY"),
)

response = client.responses.create(
    model="doubao-seed-2-1-pro-260628",
    input=[{"role": "user", "content": "What's the weather like in Beijing?"}],
    tools=[
        {
            "type": "web_search",
            "max_keyword": 2,
        }
    ],
)

for item in response.output:
    if item.type == "message":
        for content in item.content:
```

```python
if content.type == "output_text":
    print(content.text)
```

- 工具调用：学习如何让模型使用内置工具，如网页搜索、知识库检索、豆包助手等能力。
- Function Calling（函数调用）：学习如何让模型调用自定义的工具。
- 云部署 MCP / Remote MCP：学习如何让模型使用 MCP 服务。

5 下一步

现在你已经完成了首次方舟模型服务的 API 调用，你可以探索模型的更多能力，包括：

- 模型列表：快速浏览方舟提供的模型全集以及各个模型所具备的能力，快速根据你的实际场景匹配到合适的模型。

版权所有©北京火山引擎科技有限公司